AI Model Gateway Customer Guide

A simple reference for customers who want a multi-model AI platform direction, but may not have a large technical team yet.

Audience	Business and product teams with basic technical awareness
Prototype	Local AI Model Gateway with Qwen / DashScope first
Main idea	One customer API, many model providers behind the scenes
Demo mode	Mock mode first, live Qwen mode only when needed

Executive Summary

An AI Model Gateway lets customers call one simple API while the platform manages model routing, provider differences, usage control, and future fallback logic. The first prototype uses Alibaba Cloud Model Studio / Qwen because it is available now and keeps testing cost low.

- Start with Qwen and mock mode before adding more paid providers.
- Use an OpenAI-compatible API so customers can understand the integration quickly.
- Show routing visually: smart-fast is a customer-facing name that maps to qwen-plus.
- Keep enterprise API Gateway features optional until customer demand is clear.

Layered Architecture

Layer	What It Means	Example
1. Customer Systems	The customer's app, backend, or agent	Website, chatbot, internal tool
2. One Simple API	One standard API format for customers	/v1/chat/completions
3. Gateway Entry Control	Controls who can call and how much they can use	API keys, limits, logs
4. Model Control	Chooses the public model and route	smart-fast -> qwen-plus
5. Provider Adapters	Translates request and response formats	DashScope adapter
6. Model Providers	The real upstream AI model provider	Alibaba Cloud Qwen

How Route Simulation Works

The dashboard explains routing without spending money. In mock mode, the gateway reads the requested model, checks the registry, maps the public model to a real upstream model, and returns a simulated response with a route trace.

```
Customer request: model = smart-fast
Registry lookup: smart-fast maps to qwen-plus
Provider adapter: prepare DashScope-compatible request
Mock response: show the route trace without calling Alibaba Cloud
```

What The Current Prototype Includes

- Visual dashboard at the gateway root URL.
- OpenAI-compatible /v1/chat/completions endpoint.
- Customer API key authentication with Bearer token.
- Basic in-memory request limit.
- Model registry in model_registry.json.
- SQLite request and usage records for restart-safe demo data.
- Provider, customer, model, and request summary endpoints.
- Provider health endpoint for mock-ready, live-ready, degraded, and not-ready states.
- Customer reports endpoint for request count, errors, token usage, and budget state.
- Request activity endpoint with simple filters for troubleshooting.
- Config check endpoint for demo keys and missing production settings.
- Request-level fallback controls for routing demos.
- Mock OpenAI-style tool call response for agent demos.
- Simple customer plans with token and cost budgets.
- Bring Your Own Key mapping through environment variables.
- Provider adapter scaffolds for OpenAI-compatible APIs and Claude-style APIs.
- Automated mock regression test that does not spend provider credits.
- Mock mode for demos and planning.
- Live Qwen mode when DASHSCOPE_API_KEY is available.

Main Technical Difficulties

Difficulty	Why It Matters	Prototype Status
Provider format differences	Each provider may use different request and response details.	Handled first for DashScope-compatible chat
Streaming	Chat UIs and agents often need incremental tokens.	Mock streaming included; live provider differences
Tool calling	OpenAI, Claude, and Qwen may differ in tool format.	Mock tool_calls plus basic normalization
Token usage	Billing and quota require reliable usage data.	Stored in SQLite
Provider health	Customers need to know whether a provider can serve traffic.	Readiness view based on config and recent traffic
Customer reporting	Customers need a simple usage and budget story.	Per-customer report endpoint and dashboard card
Request troubleshooting	Support teams need to see what happened to a request.	Filtered request activity feed
Fallback	Retrying another model needs clear business rules.	Registry fallback plus request-level controls
Customer key management	Each customer needs limits, logs, and access control.	Minimum version plus BYOK mapping
Cost control	Different models have different prices and limits.	Simple token and cost budgets included

Why The Test Script Matters

The repository includes test_gateway.py. It starts the gateway in mock mode and checks API keys, model listing, route tracing, fallback, model access rules, token budget blocking, SQLite logs, and status output without leaking provider keys.

Admin Summary Endpoints

The prototype exposes local admin JSON views for provider status, provider health, customer reports, request activity, usage by customer, usage by model, and request summaries. These make the control layer easier to explain. The local demo uses a separate admin key. In production, this key should be changed and protected.

Provider Health

/v1/gateway/provider-health answers a simple question: can this provider serve traffic now? It checks enabled models, live key readiness, customer BYOK readiness, recent errors, and average latency. In mock mode, ready_mock means the provider can be explained without spending credits. It does not mean the live provider key is ready.

Customer Reports

/v1/gateway/customer-reports shows one report per customer. It includes request count, error count, token usage, remaining budget, models used, providers used, and recent requests. This helps explain that a gateway is also a control and reporting layer, not only a model router.

Request Activity

/v1/gateway/request-activity shows recent gateway decisions. It can filter by customer, model, provider, error code, status, and limit. This helps explain which customer sent a request, which public model was requested, which upstream model was used, which provider handled it, and whether the request succeeded or failed.

Config Check

The prototype includes /v1/gateway/config-check. It warns about demo admin keys, demo customer keys, missing live provider keys, and direct secret values in local JSON files. This is not a full security audit, but it is a useful readiness checklist for customer demos.

Recommended Roadmap

Phase	Goal	Result
1. Current prototype	Qwen-only gateway with mock dashboard	Customer can understand the concept
2. Live Qwen demo	Use Model Studio API key for real chat	Validate latency and response quality
3. Gateway controls	Persist logs, usage, model access rules, budgets, and BYOK mapping	Better customer management
4. More providers	Enable OpenAI, Claude, Xiaomi adapters	More complete OpenRouter-like direction
5. Production	Database, secrets, streaming, fallback, billing	Enterprise-ready platform path

Reference Documents

OpenRouter API Reference: https://openrouter.ai/docs/api/reference/overview/

OpenRouter Authentication: https://openrouter.ai/docs/api-keys

OpenRouter Limits: https://openrouter.ai/docs/api-reference/limits/

OpenRouter Models API: https://openrouter.ai/docs/api/api-reference/models/get-models

OpenRouter Model Fallbacks: https://openrouter.ai/docs/guides/routing/model-fallbacks

OpenRouter Provider Routing: https://openrouter.ai/docs/guides/routing/provider-selection/

OpenRouter BYOK: https://openrouter.ai/docs/use-cases/byok/

Alibaba Cloud Model Studio DashScope API Reference: https://www.alibabacloud.com/help/doc-detail/3016809.html

Final Recommendation

Start small and explain clearly. Use the visual dashboard to show the customer-facing API, the routing layer, and the provider adapter. Use mock mode for discussion and live Qwen mode only when real model testing is needed. Add more providers only after the customer understands the value of the gateway.